Native Kubernetes API Resources

A beginner-friendly briefing with one real-life analogy and one technical example for each resource.

Fundamental → Intermediate → Advanced

70 resource slides • 35 API resources • simple YAML examples

Learning map

Resources are sorted by beginner learning value, not by official Kubernetes API group.

Fundamental

1. Namespaces
2. Nodes
3. Pods
4. ReplicaSets
5. Deployments
6. Services
7. ConfigMaps
8. Secrets
9. PersistentVolumes
10. StorageClasses
11. PersistentVolumeClaims
12. ServiceAccounts
13. Roles
14. RoleBindings
15. Events

Intermediate

1. DaemonSets
2. StatefulSets
3. Jobs
4. CronJobs
5. Ingresses
6. IngressClasses
7. NetworkPolicies
8. ResourceQuotas
9. LimitRanges
10. HorizontalPodAutoscalers
11. PodDisruptionBudgets
12. ClusterRoles
13. ClusterRoleBindings
14. EndpointSlices
15. Endpoints — deprecated

Advanced

1. PriorityClasses
2. RuntimeClasses
3. Leases
4. ReplicationControllers
5. CustomResourceDefinitions

Fundamental

Core resources every Kubernetes learner should understand first.

15 API resources

FUNDAMENTAL

Namespaces

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like separate apartments inside one building. Each family has its own space, even though everyone uses the same building infrastructure.

WHAT IT IS IN KUBERNETES

A logical boundary that groups namespaced resources such as Pods, Services, ConfigMaps, Secrets, Roles, and PVCs.

ADVANTAGE IT BRINGS

Keeps teams and environments separated, supports scoped access control, and makes cleanup easier.

BEGINNER MEMORY

Namespace = project folder inside the cluster.

Think: Namespace = project folder inside the cluster.

FUNDAMENTAL

Namespaces — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: demo
```

Short description

Creates a namespace called demo.

What this configuration does

- Creates a logical workspace named demo
- Lets you place namespaced resources inside demo
- Gives RBAC and quotas a clear scope

It does not do

- Create a new Kubernetes cluster
- Isolate nodes, CPU, or network by itself
- Automatically secure all workloads

FUNDAMENTAL

Nodes

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like physical or virtual workers in a factory. They provide the floor space and machines where work actually happens.

WHAT IT IS IN KUBERNETES

A worker machine registered in the Kubernetes cluster. Nodes run Pods through kubelet and a container runtime.

ADVANTAGE IT BRINGS

Gives Kubernetes the compute capacity needed to schedule and run containers across machines.

BEGINNER MEMORY

Node = machine that runs Pods.

Think: Node = machine that runs Pods.

FUNDAMENTAL

Nodes — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: v1
kind: Node
metadata:
  name: worker-01
  labels:
    nodepool: general
```

Short description

Shows a Node object with a label that can be used for scheduling or inventory.

What this configuration does

- Represents a machine named worker-01
- Adds a nodepool=general label
- Can be selected by node selectors or affinity

It does not do

- Normally get created manually by app teams
- Start an application by itself
- Guarantee the node is healthy

FUNDAMENTAL

Pods

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a lunchbox containing one meal set. The items inside travel together, are opened together, and are thrown away together.

WHAT IT IS IN KUBERNETES

The smallest deployable workload unit. A Pod wraps one or more tightly coupled containers that share network and storage.

ADVANTAGE IT BRINGS

Gives Kubernetes a standard unit to schedule, restart, monitor, and connect containers.

BEGINNER MEMORY

Pod = one running copy of your app.

Think: Pod = one running copy of your app.

FUNDAMENTAL

Pods — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

Short description

Creates one Pod running one nginx container.

What this configuration does

- Runs one nginx container image
- Exposes container port 80 inside the Pod
- Adds app=nginx label for selection

It does not do

- Keep multiple replicas automatically
- Perform rolling updates
- Expose the app outside the cluster

FUNDAMENTAL

ReplicaSets

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a restaurant manager who keeps exactly three cashiers on duty. If one leaves, another is called in.

WHAT IT IS IN KUBERNETES

A controller that keeps a requested number of identical Pods running using labels and selectors.

ADVANTAGE IT BRINGS

Provides self-healing replicas so the app has the desired number of Pod copies.

BEGINNER MEMORY

ReplicaSet = replica counter and repairer.

Think: ReplicaSet = replica counter and repairer.

FUNDAMENTAL

ReplicaSets — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

Short description

Keeps three nginx Pods with label app=nginx running.

What this configuration does

- Creates Pods from the template
- Maintains 3 matching replicas
- Replaces Pods that disappear

It does not do

- Perform Deployment-style rolling updates
- Keep history for rollback
- Expose Pods through a stable address

FUNDAMENTAL

Deployments

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a restaurant chain manager who can open three branches, upgrade their menu gradually, and roll back if customers complain.

WHAT IT IS IN KUBERNETES

A higher-level controller for stateless apps. It manages ReplicaSets and supports rollout, rollback, and scaling.

ADVANTAGE IT BRINGS

Makes application releases safer through declarative desired state and rolling updates.

BEGINNER MEMORY

Deployment = safe release manager for stateless apps.

Think: Deployment = safe release manager for stateless apps.

FUNDAMENTAL

Deployments — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

Short description

Runs three nginx web Pods and lets Kubernetes manage rollout behavior.

What this configuration does

- Maintains 3 web Pod replicas
- Creates and owns ReplicaSets
- Supports rolling update and rollback

It does not do

- Provide stable database identity
- Expose the app without a Service
- Store application data by itself

FUNDAMENTAL

Services

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a reception desk number. People call one number even if the actual staff behind the desk changes.

WHAT IT IS IN KUBERNETES

A stable network abstraction that routes traffic to matching Pods by label.

ADVANTAGE IT BRINGS

Gives applications a fixed DNS name and virtual IP even when Pods are replaced.

BEGINNER MEMORY

Service = stable phone number for changing Pods.

Think: Service = stable phone number for changing Pods.

FUNDAMENTAL

Services — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

Short description

Creates an internal ClusterIP Service that sends traffic to Pods labeled app=web.

What this configuration does

- Provides stable DNS: web-service.default.svc
- Routes port 80 to selected Pods
- Decouples clients from Pod IP changes

It does not do

- Create Pods by itself
- Expose externally unless type/Ingress supports it
- Encrypt traffic automatically

FUNDAMENTAL

ConfigMaps

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a restaurant recipe card. The kitchen can change seasoning instructions without rebuilding the oven.

WHAT IT IS IN KUBERNETES

A namespaced object for non-sensitive configuration values such as URLs, feature flags, or text files.

ADVANTAGE IT BRINGS

Separates configuration from container images so the same image can run in different environments.

BEGINNER MEMORY

ConfigMap = non-secret settings.

Think: ConfigMap = non-secret settings.

FUNDAMENTAL

ConfigMaps — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: demo
  LOG_LEVEL: info
  welcome.txt: |
    Hello from Kubernetes
```

Short description

Stores simple non-sensitive application settings.

What this configuration does

- Stores key-value config data
- Can be mounted as files or env vars
- Lets config change without rebuilding image

It does not do

- Protect passwords or private keys
- Restart Pods automatically when changed
- Validate application-specific values

FUNDAMENTAL

Secrets

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a locked envelope handed to staff. It carries sensitive values separately from public instructions.

WHAT IT IS IN KUBERNETES

A namespaced object for sensitive data such as passwords, tokens, certificates, and keys.

ADVANTAGE IT BRINGS

Avoids hardcoding credentials into images and manifests; integrates with Pod mounts and environment variables.

BEGINNER MEMORY

Secret = sensitive settings, handle carefully.

Think: Secret = sensitive settings, handle carefully.

FUNDAMENTAL

Secrets — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: dXNlcm5hbWU= # "username" in Base64
  password: bXlwYXNzd29yZA== # "mypassword" in Base64
```

Short description

Creates an Opaque Secret from readable stringData values.

What this configuration does

- Stores username and password as Secret data
- Can be mounted into Pods securely compared to plain YAML use
- Keeps credentials separate from app image

It does not do

- Make data safe if committed to Git
- Enable encryption at rest automatically everywhere
- Rotate credentials by itself

FUNDAMENTAL

PersistentVolumes

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a rented warehouse space. It exists as storage capacity before a shop decides to use it.

WHAT IT IS IN KUBERNETES

A cluster-level storage resource representing actual storage provisioned by an admin or storage system.

ADVANTAGE IT BRINGS

Separates storage infrastructure from application claims and allows controlled reuse or retention.

BEGINNER MEMORY

PV = real storage available to the cluster.

Think: PV = real storage available to the cluster.

FUNDAMENTAL

PersistentVolumes — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: local-demo
  hostPath:
    path: /mnt/demo-data
```

Short description

Defines a simple 5Gi storage volume using hostPath for learning/demo clusters.

What this configuration does

- Makes 5Gi storage available as a PV
- Uses ReadWriteOnce access mode
- Keeps data after PVC deletion because Retain is set

It does not do

- Recommended for production storage
- Automatically create a PVC
- Work across nodes with hostPath

FUNDAMENTAL

StorageClasses

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like choosing a delivery service tier: standard, express, or cold storage. Each tier has different behavior.

WHAT IT IS IN KUBERNETES

A cluster-level storage profile used for dynamic volume provisioning.

ADVANTAGE IT BRINGS

Lets users request storage without knowing the storage backend details.

BEGINNER MEMORY

StorageClass = storage menu option.

Think: StorageClass = storage menu option.

FUNDAMENTAL

StorageClasses — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete
```

Short description

Defines a storage class named fast-ssd for local/static volume learning scenarios.

What this configuration does

- Creates a named storage profile
- Delays binding until a Pod is scheduled
- Sets default reclaim behavior to Delete

It does not do

- Create storage without a compatible provisioner
- Guarantee SSD hardware exists
- Store data by itself

FUNDAMENTAL

PersistentVolumeClaims

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a shop requesting a warehouse unit: “I need 5 GB, private access.” The landlord matches it with available space.

WHAT IT IS IN KUBERNETES

A namespaced request for storage by an application.

ADVANTAGE IT BRINGS

Lets Pods ask for storage without binding directly to a specific disk implementation.

BEGINNER MEMORY

PVC = app storage request.

Think: PVC = app storage request.

FUNDAMENTAL

PersistentVolumeClaims — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-claim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 5Gi
```

Short description

Requests 5Gi of ReadWriteOnce storage from the fast-ssd StorageClass.

What this configuration does

- Asks for 5Gi persistent storage
- Can bind to a matching PV or dynamic provisioner
- Can be mounted by Pods

It does not do

- Create a Pod by itself
- Guarantee binding if no suitable storage exists
- Define backup or retention policy

FUNDAMENTAL

ServiceAccounts

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like an employee badge for an application. It identifies what the app is allowed to do inside the building.

WHAT IT IS IN KUBERNETES

An identity that Pods use when calling the Kubernetes API.

ADVANTAGE IT BRINGS

Enables least-privilege access for workloads instead of sharing human credentials.

BEGINNER MEMORY

ServiceAccount = identity for Pods.

Think: ServiceAccount = identity for Pods.

FUNDAMENTAL

ServiceAccounts — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-reader
  namespace: default
```

Short description

Creates a ServiceAccount named app-reader in the default namespace.

What this configuration does

- Creates an API identity for Pods
- Can be referenced in Pod specs
- Can receive permissions through RBAC bindings

It does not do

- Grant permissions by itself
- Represent a human user account
- Allow access outside Kubernetes automatically

FUNDAMENTAL

Roles

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a job description inside one office: “This person may read files, but only in this department.”

WHAT IT IS IN KUBERNETES

A namespaced RBAC permission set. A Role defines allowed actions on resources inside one namespace.

ADVANTAGE IT BRINGS

Supports least-privilege access for users, groups, or ServiceAccounts.

BEGINNER MEMORY

Role = namespace-level permission recipe.

Think: Role = namespace-level permission recipe.

FUNDAMENTAL

Roles — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
  - apiGroups:
      - ""
    resources:
      - pods
    verbs:
      - get
      - list
```

Short description

Defines a permission recipe for reading Pods inside the default namespace.

What this configuration does

- Allows get pods in default
- Allows list pods in default
- Can be attached using a RoleBinding

It does not do

- Create, update, or delete Pods
- Read Pod logs unless pods/log is allowed
- Access Pods in another namespace

FUNDAMENTAL

RoleBindings

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like assigning a job description to one employee in one office.

WHAT IT IS IN KUBERNETES

A namespaced RBAC binding that grants a Role or ClusterRole to a user, group, or ServiceAccount in one namespace.

ADVANTAGE IT BRINGS

Separates permission definition from who receives it.

BEGINNER MEMORY

RoleBinding = attach permissions inside one namespace.

Think: RoleBinding = attach permissions inside one namespace.

FUNDAMENTAL

RoleBindings — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-pod-reader
  namespace: default
subjects:
  - kind: ServiceAccount
    name: app-reader
    namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Short description

Grants the pod-reader Role to the app-reader ServiceAccount in default.

What this configuration does

- Connects a subject to a Role
- Grants permissions only in default
- Uses roleRef to point at pod-reader

It does not do

- Create the Role if it is missing
- Grant cluster-wide access
- Grant permissions outside the referenced Role

FUNDAMENTAL

Events

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a building logbook: “Door opened,” “alarm fired,” “delivery arrived.” It helps explain what happened.

WHAT IT IS IN KUBERNETES

Timestamped records emitted by Kubernetes components to describe notable changes, warnings, or failures.

ADVANTAGE IT BRINGS

Helps beginners troubleshoot scheduling, pulling images, restarts, and controller actions.

BEGINNER MEMORY

Event = cluster activity log entry.

Think: Event = cluster activity log entry.

FUNDAMENTAL

Events — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: events.k8s.io/v1
kind: Event
metadata:
  name: nginx-started.17f6d
  namespace: default
regarding:
  apiVersion: v1
  kind: Pod
  name: nginx-pod
  namespace: default
  uid: "12345678-1234-1234-1234-123456789abc"
reason: Started
note: Started container nginx
type: Normal
action: Started
reportingController: demo.local/learning
reportingInstance: demo-1
eventTime: "2026-07-26T10:00:00Z"
```

Short description

Shows the shape of an Event object. In real clusters, controllers usually create Events automatically.

What this configuration does

- Records an event about nginx-pod
- Identifies the involved object in regarding
- Stores reason, note, type, and time

It does not do

- Fix the problem it reports
- Persist forever as audit history
- Usually require manual creation by app teams

Intermediate

Operational resources used to expose, protect, scale, schedule, and control workloads.

15 API resources

INTERMEDIATE

DaemonSets

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like placing one security guard on every floor of a building. Every floor gets exactly the same service.

WHAT IT IS IN KUBERNETES

A controller that runs one copy of a Pod on each matching Node.

ADVANTAGE IT BRINGS

Ideal for node-level agents such as log collectors, monitoring agents, CNI plugins, or storage agents.

BEGINNER MEMORY

DaemonSet = one Pod per node.

Think: DaemonSet = one Pod per node.

INTERMEDIATE

DaemonSets — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-logger
spec:
  selector:
    matchLabels:
      app: node-logger
  template:
    metadata:
      labels:
        app: node-logger
    spec:
      containers:
        - name: logger
          image: fluent/fluent-bit:3.1
```

Short description

Runs one log-agent Pod on every schedulable matching node.

What this configuration does

- Creates a Pod per eligible node
- Adds Pods when new nodes join
- Useful for node-level tooling

It does not do

- Run a fixed replica count like Deployment
- Provide stable network identity
- Replace need for application logging design

INTERMEDIATE

StatefulSets

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like hotel rooms with fixed room numbers and private lockers. Guest in room 101 has a stable identity and storage.

WHAT IT IS IN KUBERNETES

A workload controller for stateful apps that need stable network identity, ordered rollout, and persistent storage.

ADVANTAGE IT BRINGS

Supports databases and clustered systems that cannot treat every replica as fully interchangeable.

BEGINNER MEMORY

StatefulSet = stable names + stable storage.

Think: StatefulSet = stable names + stable storage.

INTERMEDIATE

StatefulSets — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db
  replicas: 2
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: postgres
          image: postgres:16
          env:
            - name: POSTGRES_PASSWORD
              value: demo
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 5Gi
```

Short description

Creates two PostgreSQL Pods with stable ordinal names and per-Pod PVCs.

What this configuration does

- Creates db-0 and db-1 identities
- Creates separate storage claims per Pod
- Rolls out in ordered fashion

It does not do

- Make PostgreSQL production-ready alone
- Replace backup, replication, or tuning
- Expose the DB without a Service

INTERMEDIATE

Jobs

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like hiring a cleaner for one task: clean the room once, confirm it is done, then leave.

WHAT IT IS IN KUBERNETES

A controller that runs Pods until a finite task completes successfully.

ADVANTAGE IT BRINGS

Best for migrations, batch processing, imports, one-time scripts, and verification tasks.

BEGINNER MEMORY

Job = run-to-completion task.

Think: Job = run-to-completion task.

INTERMEDIATE

Jobs — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: hello
          image: busybox:1.36
          command:
            - sh
            - -c
            - echo "Hello from a Kubernetes Job"
```

Short description

Runs a short BusyBox command once and finishes.

What this configuration does

- Creates a Pod for a finite task
- Tracks completion status
- Can retry failed Pods depending on policy

It does not do

- Run continuously like a Deployment
- Schedule repeatedly by itself
- Expose a network endpoint

INTERMEDIATE

CronJobs

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like an alarm clock that tells a cleaner to work every night at 2 AM.

WHAT IT IS IN KUBERNETES

A controller that creates Jobs on a time schedule using cron syntax.

ADVANTAGE IT BRINGS

Automates scheduled work such as backups, reports, cleanup, and periodic checks.

BEGINNER MEMORY

CronJob = scheduled Job.

Think: CronJob = scheduled Job.

INTERMEDIATE

CronJobs — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - name: cleanup
              image: busybox:1.36
              command:
                - sh
                - -c
                - echo "nightly cleanup"
```

Short description

Creates a Job every day at 02:00 cluster/controller time.

What this configuration does

- Schedules recurring Jobs
- Runs the cleanup command on schedule
- Uses Job tracking for each run

It does not do

- Guarantee app-level backup consistency
- Run if the controller is unavailable forever
- Expose a service endpoint

INTERMEDIATE

Ingresses

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a building front gate that routes visitors: “/shop goes to shop desk, /api goes to API desk.”

WHAT IT IS IN KUBERNETES

A routing rule resource for HTTP/HTTPS traffic entering the cluster, implemented by an Ingress controller.

ADVANTAGE IT BRINGS

Provides host/path-based routing and centralizes external HTTP access.

BEGINNER MEMORY

Ingress = HTTP routing rulebook.

Think: Ingress = HTTP routing rulebook.

INTERMEDIATE

Ingresses — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: demo.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-service
                port:
                  number: 80
```

Short description

Routes HTTP requests for demo.example.com to the web-service Service.

What this configuration does

- Defines host/path routing
- Targets a Service backend
- Uses the nginx IngressClass

It does not do

- Work without an Ingress controller
- Create the Service or Pods
- Automatically buy DNS or TLS certificates

INTERMEDIATE

IngressClasses

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like choosing which gate company manages your building entrance: NGINX gate, Traefik gate, or cloud gate.

WHAT IT IS IN KUBERNETES

A cluster-level resource that identifies which controller should handle an Ingress.

ADVANTAGE IT BRINGS

Allows multiple Ingress controllers to coexist and be selected explicitly.

BEGINNER MEMORY

IngressClass = which Ingress controller handles it.

Think: IngressClass = which Ingress controller handles it.

INTERMEDIATE

IngressClasses — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: nginx
spec:
  controller: k8s.io/ingress-nginx
```

Short description

Defines an IngressClass named nginx for the ingress-nginx controller.

What this configuration does

- Creates a selectable IngressClass
- Lets Ingress use ingressClassName: nginx
- Clarifies controller ownership

It does not do

- Install the Ingress controller
- Expose traffic by itself
- Configure DNS or certificates

INTERMEDIATE

NetworkPolicies

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like office security rules: the cashier desk may talk to the database room, but random visitors may not.

WHAT IT IS IN KUBERNETES

A namespaced policy that controls Pod-to-Pod and external network traffic when supported by the CNI plugin.

ADVANTAGE IT BRINGS

Reduces lateral movement and enforces service-to-service network boundaries.

BEGINNER MEMORY

NetworkPolicy = firewall rules for Pods.

Think: NetworkPolicy = firewall rules for Pods.

INTERMEDIATE

NetworkPolicies — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-to-api
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: web
      ports:
        - protocol: TCP
          port: 8080
```

Short description

Allows traffic to api Pods only from web Pods on TCP port 8080.

What this configuration does

- Selects Pods labeled app=api
- Allows ingress from app=web Pods
- Limits allowed port to 8080

It does not do

- Work without NetworkPolicy-capable CNI
- Control traffic to nodes automatically
- Encrypt traffic

INTERMEDIATE

ResourceQuotas

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a monthly office budget: one department can use only a defined amount of electricity, desks, and supplies.

WHAT IT IS IN KUBERNETES

A namespace-level policy that limits aggregate resource usage and object counts.

ADVANTAGE IT BRINGS

Prevents one team or environment from consuming all cluster resources.

BEGINNER MEMORY

ResourceQuota = namespace budget.

Think: ResourceQuota = namespace budget.

INTERMEDIATE

ResourceQuotas — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: demo-quota
  namespace: default
spec:
  hard:
    pods: "10"
    requests.cpu: "2"
    requests.memory: 4Gi
    limits.cpu: "4"
    limits.memory: 8Gi
```

Short description

Sets maximum Pod count and CPU/memory request/limit totals for default namespace.

What this configuration does

- Limits namespace to 10 Pods
- Caps total requested CPU and memory
- Caps total CPU and memory limits

It does not do

- Set defaults for individual containers
- Reserve physical hardware by itself
- Replace monitoring and capacity planning

INTERMEDIATE

LimitRanges

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like office rules saying every new desk must have a minimum and maximum size.

WHAT IT IS IN KUBERNETES

A namespace-level policy that sets default, minimum, and maximum resource values for Pods or containers.

ADVANTAGE IT BRINGS

Prevents tiny or huge containers and helps ResourceQuota work predictably.

BEGINNER MEMORY

LimitRange = default/min/max per workload.

Think: LimitRange = default/min/max per workload.

INTERMEDIATE

LimitRanges — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: container-limits
  namespace: default
spec:
  limits:
    - type: Container
      default:
        cpu: 500m
        memory: 512Mi
      defaultRequest:
        cpu: 100m
        memory: 128Mi
      max:
        cpu: "1"
        memory: 1Gi
      min:
        cpu: 50m
        memory: 64Mi
```

Short description

Defines default, minimum, and maximum container CPU/memory in default namespace.

What this configuration does

- Applies default requests/limits when omitted
- Rejects containers outside min/max bounds
- Improves predictable scheduling

It does not do

- Scale Pods automatically
- Change existing Pods retroactively
- Guarantee application performance

INTERMEDIATE

HorizontalPodAutoscalers

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like opening more checkout counters when the queue gets long, then closing them when the store is quiet.

WHAT IT IS IN KUBERNETES

A controller configuration that scales workload replicas horizontally based on metrics.

ADVANTAGE IT BRINGS

Improves elasticity and reduces manual scaling for variable traffic.

BEGINNER MEMORY

HPA = automatic replica adjuster.

Think: HPA = automatic replica adjuster.

INTERMEDIATE

HorizontalPodAutoscalers — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

Short description

Scales the web Deployment between 2 and 10 replicas to target 60% average CPU utilization.

What this configuration does

- Watches CPU utilization metric
- Adjusts Deployment replica count
- Keeps replicas within min/max range

It does not do

- Work without metrics pipeline
- Scale nodes directly
- Fix inefficient application code

INTERMEDIATE

PodDisruptionBudgets

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a hospital rule: during maintenance, at least two nurses must remain on duty.

WHAT IT IS IN KUBERNETES

A policy that limits voluntary disruptions so enough Pods remain available during maintenance.

ADVANTAGE IT BRINGS

Protects availability during node drains, upgrades, and controlled evictions.

BEGINNER MEMORY

PDB = minimum availability during voluntary disruption.

Think: PDB = minimum availability during voluntary disruption.

INTERMEDIATE

PodDisruptionBudgets — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: web
```

Short description

Requires at least two app=web Pods to remain available during voluntary disruptions.

What this configuration does

- Applies to Pods labeled app=web
- Blocks voluntary evictions below minAvailable
- Helps safer node maintenance

It does not do

- Stop involuntary crashes or hardware failure
- Create extra replicas automatically
- Guarantee zero downtime alone

INTERMEDIATE

ClusterRoles

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like a company-wide job description: “Security auditor may read all office logs across every branch.”

WHAT IT IS IN KUBERNETES

A cluster-scoped RBAC permission set that can cover cluster resources or namespaced resources across namespaces.

ADVANTAGE IT BRINGS

Centralizes reusable permissions for platform-wide operations.

BEGINNER MEMORY

ClusterRole = cluster-wide permission recipe.

Think: ClusterRole = cluster-wide permission recipe.

INTERMEDIATE

ClusterRoles — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: namespace-reader
rules:
  - apiGroups:
      - ""
    resources:
      - namespaces
    verbs:
      - get
      - list
      - watch
```

Short description

Defines permission to read Namespace resources across the cluster.

What this configuration does

- Allows get/list/watch namespaces
- Can be bound with ClusterRoleBinding
- Can also be reused inside a RoleBinding

It does not do

- Grant access until it is bound
- Allow creating or deleting namespaces
- Limit itself to one namespace

INTERMEDIATE

ClusterRoleBindings

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like assigning a company-wide job description to one employee across all branches.

WHAT IT IS IN KUBERNETES

A cluster-scoped binding that grants a ClusterRole to users, groups, or ServiceAccounts across the cluster.

ADVANTAGE IT BRINGS

Makes platform-wide access explicit and auditable.

BEGINNER MEMORY

ClusterRoleBinding = attach cluster-wide permissions.

Think: ClusterRoleBinding = attach cluster-wide permissions.

INTERMEDIATE

ClusterRoleBindings — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: bind-namespace-reader
subjects:
  - kind: ServiceAccount
    name: app-reader
    namespace: default
roleRef:
  kind: ClusterRole
  name: namespace-reader
  apiGroup: rbac.authorization.k8s.io
```

Short description

Grants namespace-reader ClusterRole to the app-reader ServiceAccount.

What this configuration does

- Connects app-reader to namespace-reader
- Grants the ClusterRole cluster-wide
- Uses a ServiceAccount as the subject

It does not do

- Create the ServiceAccount automatically
- Limit access to only default namespace
- Grant permissions not listed in the ClusterRole

INTERMEDIATE

EndpointSlices

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like splitting a large phone directory into smaller pages so updates are faster and easier to handle.

WHAT IT IS IN KUBERNETES

A scalable API that stores network endpoints backing Services. Usually created automatically for Services.

ADVANTAGE IT BRINGS

Improves scalability over old Endpoints by spreading endpoints across multiple slice objects.

BEGINNER MEMORY

EndpointSlice = scalable backend address list.

Think: EndpointSlice = scalable backend address list.

INTERMEDIATE

EndpointSlices — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: web-service-abc
  namespace: default
  labels:
    kubernetes.io/service-name: web-service
addressType: IPv4
ports:
  - name: http
    protocol: TCP
    port: 80
endpoints:
  - addresses:
      - 10.244.1.10
    conditions:
      ready: true
```

Short description

Represents one ready backend IP and port for the web-service Service.

What this configuration does

- Lists backend endpoint address 10.244.1.10
- Associates slice with web-service label
- Supports scalable service discovery

It does not do

- Usually need manual creation for normal Services
- Create the backend Pod
- Expose traffic by itself

INTERMEDIATE

Endpoints — deprecated

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like an old single-page phone directory. It works, but becomes heavy when the company grows.

WHAT IT IS IN KUBERNETES

The older API for Service backend addresses. It is deprecated in modern Kubernetes in favor of EndpointSlices.

ADVANTAGE IT BRINGS

Useful to understand legacy clusters and warnings, but new designs should use EndpointSlices.

BEGINNER MEMORY

Endpoints = old backend list; prefer EndpointSlice.

Think: Endpoints = old backend list; prefer EndpointSlice.

INTERMEDIATE

Endpoints — deprecated — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: web-service
  namespace: default
subsets:
  - addresses:
      - ip: 10.244.1.10
    ports:
      - name: http
        port: 80
        protocol: TCP
```

Short description

Shows the legacy Endpoints shape for one backend IP of web-service.

What this configuration does

- Maps web-service name to backend IP and port
- Supports older Service discovery behavior
- Helps read legacy manifests or warnings

It does not do

- Recommended for new designs
- Scale as well as EndpointSlices
- Create or manage Pods

Advanced

Specialized, legacy, and extensibility resources for deeper platform understanding.

5 API resources

ADVANCED

PriorityClasses

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like emergency vehicles getting priority on the road. Some work is more important when resources are limited.

WHAT IT IS IN KUBERNETES

A cluster-level scheduling priority definition. Pods reference it using priorityClassName.

ADVANTAGE IT BRINGS

Allows critical workloads to be scheduled ahead of less important ones and may trigger preemption.

BEGINNER MEMORY

PriorityClass = importance level for Pods.

Think: PriorityClass = importance level for Pods.

ADVANCED

PriorityClasses — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 100000
globalDefault: false
description: Important business workload priority
```

Short description

Creates a named priority class that Pods can reference.

What this configuration does

- Defines priority value 100000
- Can make selected Pods more important to scheduler
- Can support preemption behavior

It does not do

- Automatically apply to all Pods
- Guarantee unlimited capacity
- Replace careful quota and scheduling design

ADVANCED

RuntimeClasses

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like choosing a vehicle type for delivery: normal truck, armored truck, or sandboxed courier.

WHAT IT IS IN KUBERNETES

A cluster-level object that selects a container runtime handler for Pods.

ADVANTAGE IT BRINGS

Supports alternative runtimes such as sandboxed or specialized runtimes when configured on nodes.

BEGINNER MEMORY

RuntimeClass = which runtime runs the Pod.

Think: RuntimeClass = which runtime runs the Pod.

ADVANCED

RuntimeClasses — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: gvisor
handler: runsc
```

Short description

Defines a RuntimeClass named gvisor that points to runtime handler runsc.

What this configuration does

- Creates a selectable runtime class
- Lets Pods use runtimeClassName: gvisor
- Can enable sandboxed runtime setups

It does not do

- Install gVisor or runtime handler
- Work unless nodes support runsc
- Secure an app by itself

ADVANCED

Leases

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like borrowing a meeting room key for a short time. If you stop renewing it, someone else can take over.

WHAT IT IS IN KUBERNETES

A lightweight coordination object often used for leader election and node heartbeat-like coordination.

ADVANTAGE IT BRINGS

Enables efficient distributed coordination without constantly updating heavy objects.

BEGINNER MEMORY

Lease = temporary leadership/ownership token.

Think: Lease = temporary leadership/ownership token.

ADVANCED

Leases — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND


```yaml
apiVersion: coordination.k8s.io/v1
kind: Lease
metadata:
  name: controller-leader
  namespace: kube-system
spec:
  holderIdentity: controller-a
  leaseDurationSeconds: 15
  renewTime: "2026-07-26T10:00:00Z"
  acquireTime: "2026-07-26T09:59:45Z"
```


Short description

Represents controller-a holding a short leadership lease.

What this configuration does

- Records who currently holds the lease
- Defines how long the lease is valid
- Supports leader-election style coordination

It does not do

- Run a controller by itself
- Guarantee business-level lock correctness alone
- Usually require manual editing

ADVANCED

ReplicationControllers

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like an older supervisor who keeps workers counted but lacks modern rollout features.

WHAT IT IS IN KUBERNETES

A legacy controller that keeps a specified number of Pod replicas running. It predates ReplicaSets and Deployments.

ADVANTAGE IT BRINGS

Important for understanding older manifests; for new workloads, use Deployments or ReplicaSets.

BEGINNER MEMORY

ReplicationController = legacy replica keeper.

Think: ReplicationController = legacy replica keeper.

ADVANCED

ReplicationControllers — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 2
  selector:
    app: nginx-legacy
  template:
    metadata:
      labels:
        app: nginx-legacy
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

Short description

Keeps two legacy nginx Pods running using a simple selector.

What this configuration does

- Maintains 2 Pods matching app=nginx-legacy
- Replaces Pods that fail or are deleted
- Shows legacy controller syntax

It does not do

- Recommended for new applications
- Support modern selector expressions
- Provide Deployment rollout features

ADVANCED

CustomResourceDefinitions

Real-life analogy → Kubernetes meaning → advantage → memory hook

REAL-LIFE ANALOGY

Like adding a new official form type to a government office. After approval, people can submit that new kind of form.

WHAT IT IS IN KUBERNETES

An extension mechanism that adds new resource types to the Kubernetes API.

ADVANTAGE IT BRINGS

Lets platforms and operators manage custom domain objects using Kubernetes-style declarative APIs.

BEGINNER MEMORY

CRD = teach Kubernetes a new resource kind.

Think: CRD = teach Kubernetes a new resource kind.

ADVANCED

CustomResourceDefinitions — correct tech example

Minimal YAML plus what this specific configuration does and does not do

YAML / COMMAND

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: widgets.demo.example.com
spec:
  group: demo.example.com
  scope: Namespaced
  names:
    plural: widgets
    singular: widget
    kind: Widget
    shortNames:
      - wd
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                size:
                  type: string
```

Short description

Adds a namespaced custom API kind called Widget under group demo.example.com.

What this configuration does

- Registers widgets.demo.example.com
- Allows creating Widget resources
- Defines a basic schema for spec.size

It does not do

- Create a controller/operator automatically
- Guarantee business logic by itself
- Replace careful API versioning design

References and next steps

OFFICIAL REFERENCES

Kubernetes Documentation: kubernetes.io/docs
API Reference:
kubernetes.io/docs/reference/kubernetes-api
EndpointSlices:
kubernetes.io/docs/concepts/services-networking/endpoint-slices
Endpoints deprecation note: Kubernetes v1.33 blog

PRACTICE ORDER

1. Create a Namespace
2. Run a Pod
3. Move to Deployment + Service
4. Add ConfigMap/Secret
5. Add RBAC and storage
6. Add network, autoscaling, and availability controls

Best beginner rule: first learn what each resource owns, then learn which controller watches it, then learn what failure it protects you from.